CSC1202, 2026 – Week 4: Prompt Quality and Structured Prompt Design

- **Part 1:** Prompt Quality and Improvement
- **Part 2:** Structured Prompt Design

References: REF1: Anthropic, *Prompting Best Practices*, Claude Documentation. **REF2:** Google, *Prompt Design Strategies*, Gemini API Documentation. Additional references are provided on the relevant slides.

- **Prompt Quality and Improvement**

When prompts go wrong

Experimental evidence shows that problems in task descriptions can significantly affect model performance.

Common problems:

- **Incomplete:** information needed to determine the required task is missing
- **Ambiguous:** the task allows more than one reasonable interpretation
- **Contradictory:** requirements in the task conflict with each other

Example

- **Original:** Call the weather API to get the **current temperature in Dublin**.
- **Incomplete:** Call the weather API to get **weather information**.
- **Ambiguous:** Call the weather API to get the **temperature for a location**.
- **Contradictory:** Call the weather API to get the **current temperature in Dublin**, but **do not provide a location**.

Ref: M. Larbi et al., When Prompts Go Wrong: Evaluating Code Model Robustness to Ambiguous, Contradictory, and Incomplete Task Descriptions, ICSE 2026, https://arxiv.org/abs/2507.20439

Example — Prompt Quality and API Calls

API: transfer_money(amount, from_account, to_account)

- **Incomplete**

"Transfer €500 from my current account."
to_account is missing.

- **Ambiguous**

"Transfer €500 from my current account to John's account."
The user has two saved beneficiaries named John. to_account could refer to either account.

- **Contradictory**

"Transfer €500 to John's account, but do not transfer more than €300."
The requested amount conflicts with the stated maximum.

Why Do These Problems Matter?

The model may **still respond or act** even when the request is not well specified:

- **A plausible response does not mean the request was correctly understood.**

- **Incomplete:** may **invent missing information** *(missing API value: **the model generates a value**)*
- **Ambiguous:** may **choose the wrong interpretation** *(two possible accounts: **the model selects the wrong one**)*
- **Contradictory:** may **satisfy one requirement but violate another** *(meeting before 12:00, but user available only after 14:00 : **schedules a time that violates one constraint**)*

- This can lead to **wrong answers, incorrect API calls, or unintended actions**.

Quiz 1

For each request, classify it as **Incomplete, Ambiguous, or Contradictory**.

1. API: schedule_meeting(person, date, start_time, duration)
Request: "Schedule a meeting with Sarah tomorrow at 10:00 AM."

2. API: send_file(file, recipient)
There are two contacts named **Alex Smith**.
Request: "Send the report to Alex Smith."

3. API: book_flight(destination, arrival_time)
Request: "Book a flight to Paris that arrives before 10:00 AM, but not before 12:00 PM."

4. API: cancel_booking(booking_id)
The user has two bookings on Friday: a **hotel** and a **restaurant**.

Request: "Cancel my booking for Friday."
API: order_product(product, quantity, max_price)
Request: "Order 3 wireless keyboards for a reasonable price."

Quiz 1: Answers

1. **Incomplete** — duration is missing.

2. **Ambiguous:** there are two possible recipients named Alex Smith.

3. **Contradictory:** the flight cannot arrive both before 10:00 AM and not before 12:00 PM.

4. **Ambiguous:** it is unclear which of the two Friday bookings should be cancelled.

Prompt quality problems and their fixes

Checklist for good quality prompt design (guide):

- **Vague instructions:** make imprecise terms or criteria more specific (response could be general, verbose, etc)
- **Ambiguous instructions:** clarify which meaning is intended
- **Missing context:** provide relevant context
- **Poor or unclear constraints:** specify clear, measurable constraints
- **Unclear output requirements:** define the expected output format
- **Unrepresentative examples:** provide examples that cover different relevant cases
- **Too many tasks in one prompt:** simplify the task or split it into separate prompts

Then: test the outputs, diagnose problems. refine the prompt, test again

Ref: Anthropic, *Prompting best practices*

Vague Instructions: Clear and Specific

Vague instruction:

Task: *Improve this Python program.*

Clear and specific instruction:

Task: *Review the Python program and identify code that may cause unnecessary execution time. Suggest up to 3 changes that could improve its performance without changing its behaviour.*

Output: *For each change, show the relevant code and briefly explain the improvement.*

- **"Improve" is vague because it is not precise enough:** the instruction does not say **what aspect of the program should be improved**.
- For example, improvement could refer to **performance, readability, maintainability, or correctness**.
- The revised instruction removes the vagueness by specifying **performance as the aspect to improve**.

Ambiguity → Make Requirements Explicit

API: get_transactions(account, date_range)
Ambiguous instruction

Task: Get the recent transactions from my savings account.

- **What is ambiguous?**

"Recent" can mean different periods: **today, the last 7 days, the last 30 days, etc.**

- The model may choose a **date_range** that the user did not intend.

Explicit requirement

Task: Get the transactions from my savings account for the **last 7 days**.

- **Vague:** what is required is not precise enough.
- **Ambiguous:** a value or instruction can have more than one reasonable interpretation.
- **Key point:** Make the intended interpretation explicit before calling the API.

Missing Context → Provide Relevant Context

Without relevant context

Task: Reply to this customer.
Customer: "Can I return the laptop I bought 20 days ago?"

- The task is clear, but the model **cannot give the correct answer without knowing the return policy**.

With relevant context

Context (return policy): Customers may return laptops within 30 days of purchase for a full refund.

Task: Reply to this customer.
Customer: "Can I return the laptop I bought 20 days ago?"

- **Relevant context** gives the model information it needs for the task but cannot obtain from the instruction alone.

Missing Constraints → Provide Relevant Constraints

Without the relevant constraint

Task: Book a hotel for the conference.
Input: Hilton Hotel, €210 per night.

The agent may book it because nothing in the prompt tells it otherwise.

With the relevant constraint

Task: Book a hotel for the conference.
Policy constraint: Hotel cost must not exceed €180 per night.
Request: Hilton Hotel, €210 per night.

The agent should **not book it** because it violates the constraints / policy.

- Constraints define **boundaries the model must respect**.
- Missing constraints can lead to outputs or actions that violate **policies, rules, limits, or requirements**.

Unspecified Output Requirements → Define the Expected Output

Output not specified

Task: Extract every deadline from this email.
- <email> ... </email>

The model decides how to present the result.

Output specified

Task: Extract every deadline from this email.
Output: JSON list:
{"task": "...", "due": "YYYY-MM-DD"}
If none, return [].
<email> ... </email>

- Without output requirements, a correct answer may still be **difficult to use**.
- Define the expected **format and structure**, especially when another program will use the output.

Unrepresentative Examples → Representative Examples

Unrepresentative examples

Task: Classify the sentiment as Positive, Negative, or Neutral.
Examples:
"Great battery!" → Positive
"Love the screen!" → Positive
"Very happy with it." → Positive

Representative examples

Task: Classify the sentiment as Positive, Negative, or Neutral.
Examples:
"Great battery!" → Positive
"Stopped working after a week." → Negative
"It works, nothing special." → Neutral

- Examples should **represent the different cases the model is expected to handle**.

Too Many Tasks in One Prompt → Break Complex Work into Steps

Complex prompt:

- Search the web for recent LLM security risks, select the most important ones, organise them by category, and write a report explaining each risk with supporting references.

Break the work into steps

- **Prompt 1 — Search and select**
  Find important LLM security risks reported in recent research and provide relevant sources for each.
- **Prompt 2 — Organise**
  Group the selected risks into appropriate categories and create an outline.
- **Prompt 3 — Write**
  Using the outline and selected sources, write the report, briefly explaining each risk and citing the relevant sources.

- Combining many dependent tasks can make it **harder to identify where a failure occurred**.
- Breaking complex work into steps allows each intermediate result to be **checked before continuing**. This is often called **prompt chaining**.

Quiz 2

Task: Label each statement with the main prompt-quality problem: **Vague instruction, Missing context, Missing constraint, Unspecified output requirements, Unrepresentative examples.**

1. Evaluate whether this project proposal is sufficiently strong.

2. You have only this customer message; no external information is available:
   "Is installation included in the price?" Respond to the customer.

3. Recommend a laptop for a new employee. The company will pay for the purchase.

4. Extract the product name, price, and quantity from the customer order so the result can be passed to an order-processing system..

5. Classify support requests as **Billing, Technical, Account, or Delivery**. Examples: "Charged twice." → **Billing**; "Invoice amount is wrong." → **Billing**; "Charged after cancelling." → **Billing**.

Quiz 2: Answers

1. **Vague instruction`:** "sufficiently strong" does not define the evaluation criteria.

2. **Missing context:** the information about whether installation is included is not available.

3. **Missing constraint:** no purchasing limit or budget is specified.

4. **Unspecified output requirements:** the information to extract is specified, but not the structure required by the order-processing system.

5. **Unrepresentative examples:** four classes are defined, but the examples cover only Billing.

Improving Prompt Quality: Test → Diagnose → Refine

- **Applying the iterative prompt-engineering process from Week 3 to prompt quality**

- **Test**
  - Run the prompt on **representative inputs**
  - Check whether outputs meet the expected requirements

- **Diagnose**
  - Identify **what went wrong**
  - Check the **instruction, context, constraints, examples, output requirements, or task complexity**

- **Refine**
  - Change the part of the prompt causing the problem
  - Test again using the **same inputs**
- **Repeat until the prompt produces sufficiently reliable results**

Quiz 3

Task: An LLM is prompted to summarise meeting notes and produce:

- Key decisions
- Action items with responsible person
- Unresolved issues

Testing shows that the summaries are generally accurate, but when an action is discussed **without assigning a person**, the model often invents a responsible person.

What is the most appropriate refinement?

A. Add more detailed meeting notes as input
B. Tell the model to omit action items when no responsible person is identified
C. Tell the model to explicitly indicate when the responsible person is not specified
D. Ask the model to produce a shorter summary

Quiz 3: Answer

Answer: C

Structured Prompt Design

Using Frameworks to Guide Prompt Design

- Prompts are often written **intuitively, without following a systematic method**, making it easy to overlook important information or instructions.

- In Part 1, we identified common **prompt-quality problems** and used **test → diagnose → refine** to improve prompts.

- **Prompt frameworks** provide practical guidance for designing prompts more systematically, similar to frameworks used in requirements engineering :
  - **SMART** *(Specific, Measurable, Achievable, Relevant, Time-bound)*
  - **INVEST** *(Independent, Negotiable, Valuable, Estimable, Small, Testable)*

Examples:

- **APE** — Action, Purpose, Expectation
- **RISE** — Role, Input, Steps, Expectation
- **CLEAR** — Concise, Logical, Explicit, Adaptive, Reflective

- **Note:** Like SMART or INVEST, these frameworks are **guides rather than fixed rules**. Effective prompt design still requires **testing, evaluation, and refinement**.

APE — Action, Purpose, Expectation

APE provides three questions to consider when designing a prompt:

- **Action — What should the model do?**
  State the task clearly: *summarise, explain, compare, recommend, generate...*

- **Purpose — Why is the result needed?**
  State the **goal the result should help achieve**.

- **Expectation — What should the result look like?**
  Specify what is expected, such as **content, format, length, or level of detail**.

APE helps ensure that the prompt communicates what to do, why it is needed, and what result is expected.

APE : Example

- **Initial prompt**

Find me the best books about large language models.

- **Improving the prompt using APE**

Action: Recommend books about large language models.

Purpose: I want to buy one book to read alongside this module. My budget is limited, and I need to know what each book covers before choosing.

Expectation: Recommend at most five books. For each, provide the **title, author, year, price, topics covered, and a link**.

RISE - Role, Input, Steps, Expectation

RISE is useful for tasks involving **input material and multiple steps**.

- **Role — Who should the model act as?**
  Specify the relevant expertise or perspective, e.g., **AI expert, code reviewer, technical writer**.

- **Input — What should the model work with?**
  Specify the **information, documents, data, or other material** to use.

- **Steps — How should the task be carried out?**
  Decompose the task into subtasks and specify the steps to follow and their order.

- **Expectation — What result is expected?**
  Specify the expected **result and how it should be returned or expressed**

RISE makes explicit the model's role, the input it should use, the steps it should follow, and the expected result.

RISE : Example

- **Initial prompt**

Find me the best books about large language models.

- **Improving the prompt using RISE**

Role: You are an AI expert familiar with large language models and the relevant literature.

Input: Use the **CSC1202 module syllabus** as the basis for determining which topics the book should cover.

Steps: decompose the task into subtasks:

- Identify candidate books.
- Find information about each book from publisher or bookseller sources.
- Compare each book's coverage with the module syllabus.
- Identify important syllabus topics that each book does not cover.

Expectation: Recommend at most five books. For each, provide the **title, author, year, price, topics covered, topics not covered, and a link**.

CLEAR — Concise, Logical, Explicit, Adaptive, Reflective

CLEAR provides five criteria for **reviewing and improving prompts**.

- **Concise — Is the prompt focused?**
  Remove unnecessary information and unrelated tasks.

- **Logical — Is the prompt organised coherently?**
  Present instructions, information, and steps in a sensible order.

- **Explicit — Are the requirements clear?**
  State important requirements, constraints, and expected results explicitly.

- **Adaptive — Is the prompt producing the result you need?**
  Refine the prompt when the result is not satisfactory.

- **Reflective — How good is the response?**
  Evaluate the response for accuracy, relevance, and completeness, and use what you learn to improve future prompts

APE and RISE mainly guide what to include when constructing a prompt. CLEAR also guides how to evaluate and improve the prompt after seeing its outputs.

CLEAR – Example: Review the Prompt(1)

- **Initial prompt**

Recommend books about large language models. I want books that cover both LLM foundations and practical development. I am mainly interested in books published recently. Recommend as many as you can and tell me which ones are best for learning the subject.

- **Review using CLEAR**

Concise: The prompt is already **focused and concise**. No change is needed.

Logical: The prompt is organised reasonably: **task → desired coverage → selection preference**. No major change is needed.

Explicit: Some requirements are not precise. Replace **"recently," "as many as you can," and "best"** with explicit criteria, e.g., **published since 2024, at most five books, and compare their coverage of foundations and practical development**.

CLEAR – Example: Example: Learn from the Output (2)

Run the revised prompt and inspect the output

- Suppose the model recommends a book that **does not exist**.

Adaptive: Modify the prompt to require that each recommended book be verified using a **publisher or bookseller source**, then run it again.

Reflective: Evaluate the new response: **Are all recommended books now verifiable? Did the change solve the problem, or are further improvements needed?**

CLEAR: Improved Prompt

Recommend **at most five books about large language models published since 2024**.

The books should cover both **LLM foundations and practical development**.

For each book:

- Provide the **title, author, year, and topics covered**
- Provide a **publisher or bookseller link confirming that the book exists**
- Briefly compare its coverage of **LLM foundations and practical development**
- If a book cannot be verified, **do not recommend it**.

Quiz 4

Task:

A researcher wants an LLM to summarise an attached paper **for a research meeting**. The summary should be **no more than 200 words** and cover the **research problem, approach, and main findings**.

Structure the prompt using **APE**:

- **Action:**
- **Purpose:**
- **Expectation:**

Quiz 4: Possible answer

- **Action:** Summarise the attached research paper, covering the research problem, approach, and main findings.
- **Purpose:** Prepare a summary for a research meeting.
- **Expectation:** Produce a summary of no more than 200 words.

Task:

A prompt is used repeatedly to review project proposals: “Review this proposal carefully and provide useful feedback.”

Testing shows that the feedback varies considerably between runs: sometimes it focuses on objectives, sometimes on methodology, and sometimes on risks.

Using **CLEAR**, suggest **one change to the prompt** that could make the reviews more consistent.

Quiz 5: Possible answer

- Make the evaluation requirements **Explicit**:

For example, “Review the proposal against these three criteria: objectives, methodology, and risks. Provide feedback on each criterion.”